


**UNIVERSITÀ DI PISA**

DEPARTMENT OF INFORMATION ENGINEERING

MSc in Computer Engineering

# Advanced Computer Networking Project

Alessio Simoncini, Lorenzo Vezzani

ACADEMIC YEAR 2025/2026

# Contents


**1 Introduction** **5**
- 1.1 Aims and scope of the documentation . . . 5
- 1.2 Project description . . . 5
- 1.3 Objectives . . . 5
- 1.4 Environment . . . 6
- 1.5 Code repository . . . 6

**2 Network structure** **7**
- 2.1 Network overview . . . 7
  - 2.1.1 Base network . . . 7
  - 2.1.2 Developed network . . . 7
- 2.2 Node images . . . 10
  - 2.2.1 FRR nodes . . . 10
  - 2.2.2 Alpine nodes . . . 10
  - 2.2.3 Manager node . . . 11
- 2.3 Network details . . . 11
  - 2.3.1 Customer side . . . 11
  - 2.3.2 Core part . . . 12
  - 2.3.3 Upstream side . . . 14

**3 Implementation** **17**
- 3.1 Containerlab topology file . . . 17
  - 3.1.1 Node types . . . 17
- 3.2 Node configurations . . . 19
  - 3.2.1 FRR nodes . . . 19
  - 3.2.2 Alpine nodes . . . 20
- 3.3 Templates . . . 20
  - 3.3.1 FRR template . . . 22
  - 3.3.2 Alpine template . . . 27
- 3.4 Network automation . . . 28
  - 3.4.1 Load balancing algorithm . . . 29
  - 3.4.2 Local Preference Setting . . . 30
  - 3.4.3 Filtering method setting . . . 31
  - 3.4.4 BGP Clearing . . . 31

3.4.5 Route Maps Clearing . . . . . 32

**4 Testing** **33**
4.1 Network deploy . . . . . 33
4.2 Nodes configuration . . . . . 33
4.3 Automation testing . . . . . 37
4.3.1 Resulting tables . . . . . 39
4.3.2 Traceroute examples . . . . . 42

**5 Conclusion** **45**
5.1 Results . . . . . 45
5.2 Future work . . . . . 45

# Chapter 1

# Introduction

## 1.1 Aims and scope of the documentation

This work has been developed as part of the *Advanced Computer Networking* course project, academic year 2025/2026.

## 1.2 Project description

The project consists in the design and implementation of a network composed by a hierarchy of Autonomous Systems (AS). The purposes of the project are using templates for network provisioning, and the study of the BGP protocol, in particular eBGP, iBGP and the dynamic configuration of BGP attributes.

## 1.3 Objectives

Three main objectives are given in the project specifications:

1. **Configuration templates** — Develop reusable configuration templates for all routers in the network using Jinja2 scripting. These templates should support parameterization for device-specific details such as AS numbers, router IDs and interface addresses and BGP neighbor relationships.

2. **Network emulation with Containerlab** — Use Containerlab to instantiate the described network topology. Each router should be based on FRR, end nodes will be based on Alpine.

3. **Network Automation System for AS 65020** — Design and implement a network automation system that operates within a Manager node in AS65020, which should receive traffic prediction, determine optimal traffic distribution across upstream links and adjust BGP attributes influence inbound and outbound traffic paths.

The basic structure of the network to emulate is shown and discussed in the following chapter.

## 1.4 Environment

As requested by project specification, the environment used for carrying out the network emulation is ContainerLab. Containerlab is an open-source tool used to create virtual networking labs based on container technology. It allows users to define network topologies through YAML files; network devices, including routers, switches, and hosts, are then emulated as containers.

## 1.5 Code repository

The code developed for the project can be found in the repository at:
`https://github.com/AlessioSmn/Network-Provisioning-Automation` .

# Chapter 2

# Network structure

## 2.1 Network overview

### 2.1.1 Base network

The network is composed by a central AS (AS65020), which will be the focus of later tasks, two customer ASes, two upstream ASes and the Internet. The basic structure of the network to implement is represented in Figure 2.1:

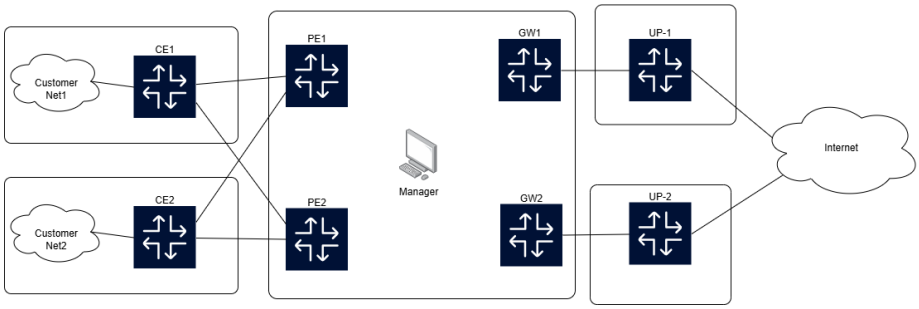



Figure 2.1: Starting network

### 2.1.2 Developed network

The network developed for the project expands the base network by specifying the structure of the various networks: customer networks, the core network and the Internet. All aspects are described in detail, along with more precise images, in the following section. The structure of the implemented network is represent in Figure 2.2, along with interface names and addresses:

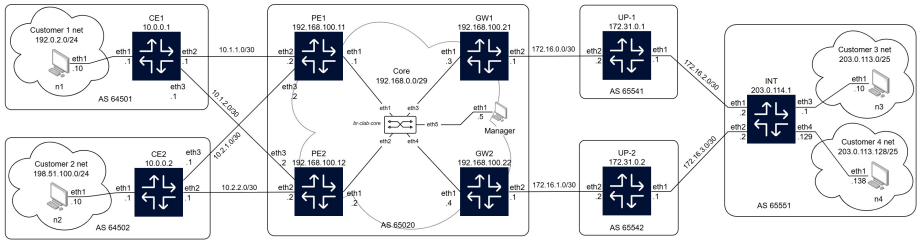



Figure 2.2: Starting network

### Management network

When deploying the network, Containerlab, based on Docker, creates all nodes as containers and automatically configures a management interface (`eth0`), assigns it an IP address, and installs a default route through that interface. To the best of our knowledge, there is no supported way to prevent the creation of this interface and its associated default route. For this reason, we decided to explicitly use it as the management interface.

**Management addresses** The management network prefix can be configured in the topology file, as shown in Listing 2.1:

```
1 mgmt:
2   network: acn_prj_mgmt               # name
3   ipv4-subnet: 10.255.255.0/24        # ipv4 range
```

Listing 2.1: Management network

By default, Docker assigns management IP addresses dynamically. However, fixed IP addresses can be explicitly specified for each node, as shown in Listing 2.2. This configuration is particularly useful for debugging purposes, as it allows consistent addressing across multiple deployments.

```
1     PE1:
2       group: frr_nodes
3       mgmt-ipv4: 10.255.255.111
```

Listing 2.2: Management IP address

**Management VRF** Another possible source of problem is the default route towards the management interface, which is installed as a route of type `kernel`, and thus cannot be removed.
To avoid unintended traffic being forwarded toward the management network, we configured the startup scripts of all nodes to optionally move the management interface into a dedicated and isolated VRF. To allow us to avoid that default route and consequent traffic towards the management network, we configured the startup files of all nodes to be able to move the management interface into

a dedicated and separated VRF. If the environment variable `CLAB_MGMT_VRF` is specified in the topology file, as represented in Listing 2.3,

```
topology:
  groups:
    frr_nodes: # applied to all nodes of the group
      env:
        CLAB_MGMT_VRF: MGMT
        CLAB_MGMT_GW: 10.255.255.1
```

Listing 2.3: Management VRF in acn.clab.yml

the startup script receives this environment variable and, when defined, creates the management VRF, moves the `eth0` interface into it and moves the default route, as represented in Listing 2.4:

```
# VRF Management (if specified)
if [ -n "${CLAB_MGMT_VRF}" ]; then

    # VRF creation
    ip link add "${CLAB_MGMT_VRF}" type vrf table 100
    ip link set "${CLAB_MGMT_VRF}" up

    # Interface assigned to VRF
    ip link set eth0 master "${CLAB_MGMT_VRF}"

    # Default route moved
    ip route add default via ${CLAB_MGMT_GW} vrf "${CLAB_MGMT_VRF}"
fi
```

Listing 2.4: Management VRF in .clab.yml

In the following table the management network IP addresses of each node are written down:

| Node | Address |
|---|---|
| CE1 | 10.255.255.101 |
| CE2 | 10.255.255.102 |
| PE1 | 10.255.255.111 |
| PE2 | 10.255.255.112 |
| GW1 | 10.255.255.121 |
| GW2 | 10.255.255.122 |
| UP1 | 10.255.255.131 |
| UP2 | 10.255.255.132 |
| INT | 10.255.255.141 |
| n1 | 10.255.255.201 |
| n2 | 10.255.255.202 |
| n3 | 10.255.255.203 |
| n4 | 10.255.255.204 |
| MNGR | 10.255.255.205 |

Table 2.1: Management IP addresses

## 2.2 Node images

Three types of nodes are present in the network: `FRR` routers, `Alpine` hosts and a single `Manager` node. All nodes are based on a custom-built images in order to at least have the OpenSSH package: all nodes are configured to be accessible via SSH via their management IP address, by using:

- username: `root`
- password: `admin`

### 2.2.1 FRR nodes

Based on FRRouting image, or FRR, version `10.4.1`. OpenSSH is added, username and password are set, and keys are generated. The final image is build by the `/shell/images.sh` script and named `frr-ssh:10.4.1`.
The FRR dockerfile is found at `./config/frr/dockerfile`.

### 2.2.2 Alpine nodes

Based on an Alpine image, version `3.19.1`. OpenSSH is added, username and password are set, and keys are generated. The final image is build by the `/shell/images.sh` script and named `alpine-ssh:3.19.1`.
The Alpine dockerfile is found at `./config/alpine/dockerfile`.

### 2.2.3 Manager node

Based on an Alpine image, version `3.19.1`. Most importantly OpenSSH and Paramiko are added: username and password are set, and keys are generated to enable SSH access, while Paramiko is needed to be able to execute Python scripts on the node. The final image is build by the `/shell/images.sh` script and named `python-ssh:3.12-alpine`.
The Manager dockerfile is found at `./config/mngr/dockerfile`.

## 2.3 Network details

### 2.3.1 Customer side

Each customer network in the base structure has been implemented with a simple point to point link between the Customer Edge router (CE1 / CE2) and the relative end host (n1 / n2).

#### Customer 1 addresses

The customer 1 network has been given the **ASN 64501**.
The customer 1 part of the network is shown in detail in Figure 2.3:

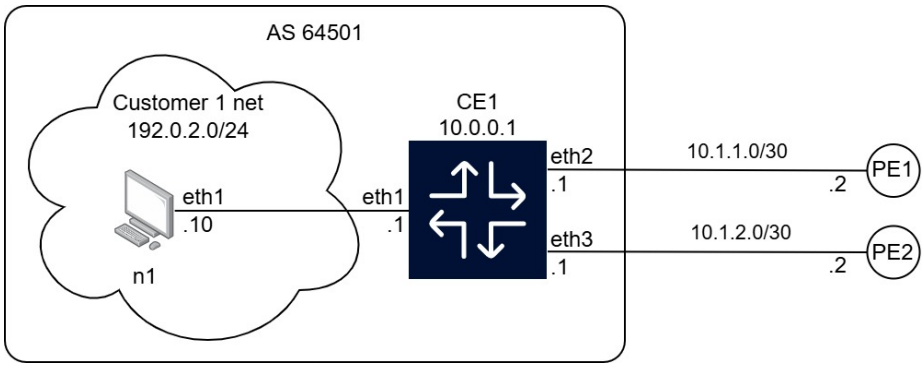



Figure 2.3: Network - customer 1

- `CE1 loopback address`: 10.0.0.1/32 (lo)
- `n1 - CE1 P2P link`: 192.0.2.0/24
  - `n1`: 192.0.2.10 (eth1)
  - `CE1`: 192.0.2.1 (eth1)
- `CE1 - PE1 P2P link`: 10.1.1.0/30
  - `CE1`: 10.1.1.1 (eth2)
  - `PE1`: 10.1.1.2 (eth2)

- `CE1 - PE2 P2P link`: 10.1.2.0/30
  - `CE1`: 10.1.2.1 (eth3)
  - `PE2`: 10.1.2.2 (eth3)

**Customer 2 addresses**

The customer 2 network has been given the **ASN 64502**.
The customer 2 part of the network is shown in detail in Figure 2.4:

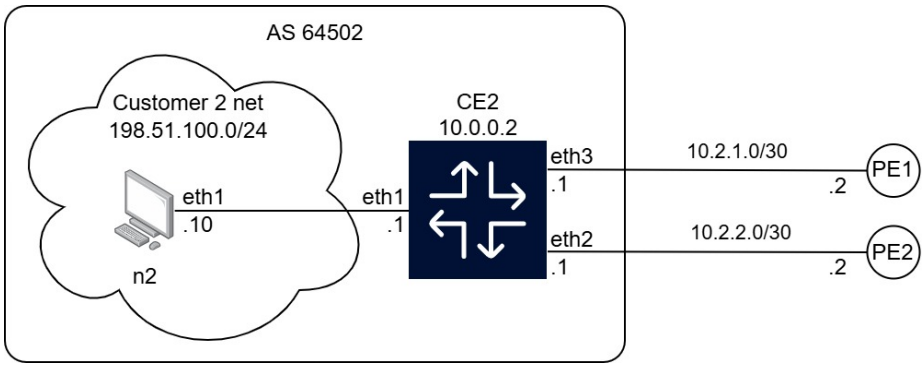



Figure 2.4: Network - customer 2

- `CE2 loopback address`: 10.0.0.2/32 (lo)
- `n2 - CE2 P2P link`: 198.51.100.0/24
  - `n2`: 198.51.100.10 (eth1)
  - `CE2`: 198.51.100.1 (eth1)
- `CE2 - PE1 P2P link`: 10.2.1.0/30
  - `CE2`: 10.2.1.1 (eth3)
  - `PE1`: 10.2.1.2 (eth3)
- `CE2 - PE2 P2P link`: 10.2.2.0/30
  - `CE2`: 10.2.2.1 (eth2)
  - `PE2`: 10.2.2.2 (eth2)

### 2.3.2 Core part

The core of the AS 65020 is implemented as a single LAN, using a switch called `br-clab-core` in the topology file; the LAN IP prefix is `192.168.0.0/29`. The central part of the network is shown in detail in Figure 2.5:

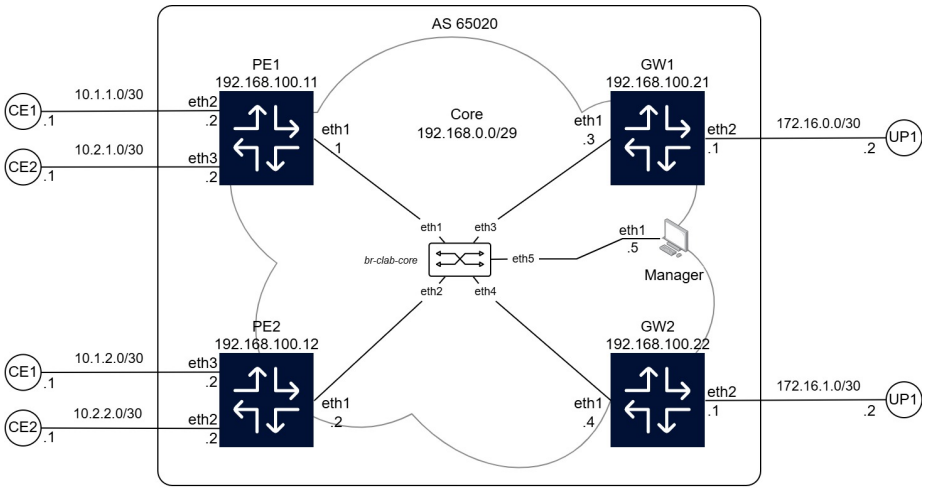



Figure 2.5: Network - central part

### Core addresses

- `Core LAN`: 192.168.0.0/29
  - `PE1`: 192.168.0.1 (eth1)
  - `PE2`: 192.168.0.2 (eth1)
  - `GW1`: 192.168.0.3 (eth1)
  - `GW2`: 192.168.0.4 (eth1)
  - `MNGR`: 192.168.0.5 (eth1)

### PE1

- `PE1 loopback address`: 192.168.100.11/32 (lo)
- `CE1 - PE1 P2P link`: 10.1.1.0/30
  - `CE1`: 10.1.1.1 (eth2)
  - `PE1`: 10.1.1.2 (eth2)
- `CE2 - PE1 P2P link`: 10.2.1.0/30
  - `CE2`: 10.2.1.1 (eth3)
  - `PE1`: 10.2.1.2 (eth3)

**PE2**

- `PE2 loopback address`: 192.168.100.12/32 (lo)
- `CE1 - PE2 P2P link`: 10.1.1.0/30
  - `CE1`: 10.1.2.1 (eth3)
  - `PE2`: 10.1.2.2 (eth3)
- `CE2 - PE2 P2P link`: 10.2.1.0/30
  - `CE2`: 10.2.2.1 (eth2)
  - `PE2`: 10.2.2.2 (eth2)

**GW1**

- `GW1 loopback address`: 192.168.100.21/32 (lo)
- `GW1 - UP1 P2P link`: 172.16.0.0/30
  - `GW1`: 172.16.0.1 (eth2)
  - `UP1`: 172.16.0.2 (eth2)

**GW2**

- `GW2 loopback address`: 192.168.100.21/32 (lo)
- `GW2 - UP2 P2P link`: 172.16.1.0/30
  - `GW2`: 172.16.1.1 (eth2)
  - `UP2`: 172.16.1.2 (eth2)

### 2.3.3 Upstream side

The internet network represented in the base network is implemented via a router, called `INT`, connected to both upstream routers and then connected to two hosts (n3 and n4): these addiotional networks and hosts have been created to be able to properly test load balancing on upstream links, by having many possible destination networks.
The upstream 1 network (router) has been given the **ASN 65541**; The upstream 2 network (router) has been given the **ASN 65542**; The Internet networks have been given the **ASN 65551**.
The upstream side of the network is shown in detail in Figure 2.6:

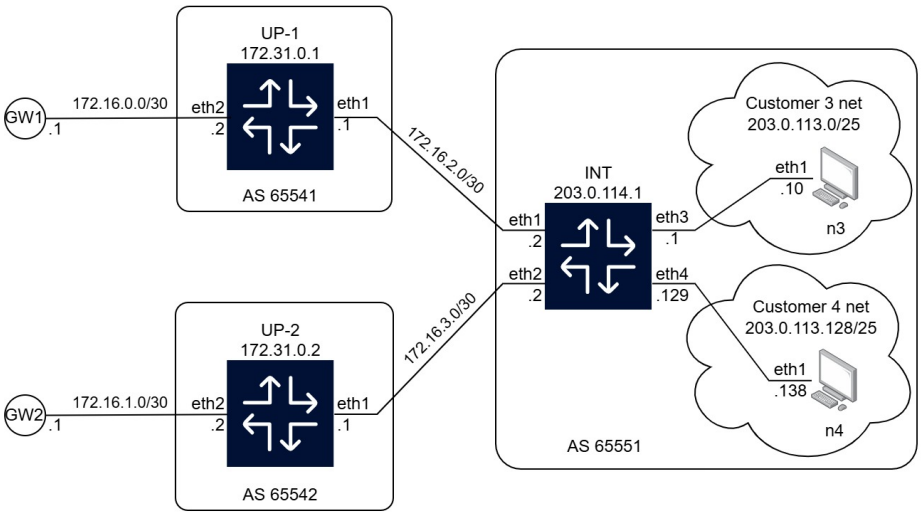



Figure 2.6: Network - upstream side

**Upstreams**

- `UP1 loopback address`: 172.31.0.1/32 (lo)
- `UP2 loopback address`: 172.31.0.2/32 (lo)
- `GW1 - UP1 P2P link`: 172.16.0.0/30
  - `GW1`: 172.16.0.1 (eth2)
  - `UP1`: 172.16.0.2 (eth2)
- `GW2 - UP2 P2P link`: 172.16.1.0/30
  - `GW2`: 172.16.1.1 (eth2)
  - `UP2`: 172.16.1.2 (eth2)
- `UP1 - INT P2P link`: 172.16.2.0/30
  - `UP1`: 172.16.2.1 (eth1)
  - `INT`: 172.16.2.2 (eth1)
- `UP2 - INT P2P link`: 172.16.3.0/30
  - `UP2`: 172.16.3.1 (eth1)
  - `INT`: 172.16.3.2 (eth2)

**Internet**

- `INT loopback address`: 203.0.114.1/32 (lo)
- `INT - n3 P2P link`: 203.0.113.0/25
  - `INT`: 203.0.113.1 (eth3)
  - `n3`: 203.0.113.10 (eth1)
- `INT - n4 P2P link`: 203.0.113.128/25
  - `INT`: 203.0.113.129 (eth4)
  - `n4`: 203.0.113.138 (eth1)

# Chapter 3

# Implementation

## 3.1 Containerlab topology file

The topology file is named `acn.clab.yml`, and it's located in the root of the project directory (`./acn.clab.yml`).

```
topology:

  groups:
    frr_nodes:
      # ...
    alp_nodes:
      # ...
    pyt_nodes:
      # ...
    bridges:
      kind: bridge

  nodes:
    # All previously described nodes
    # ...
    PE1:
      group: frr_nodes
      mgmt-ipv4: 10.255.255.111
    # ...

  links:
    # List of all links
```

Listing 3.1: acn.clab.yml structure

The list of links present in the topology file implements the connections described in section 2.3.

### 3.1.1 Node types

To implement the three types of nodes discussed in previous sections (FRR, Alpine and Alpine-Python) three node types have been defined in the network

topology: the keyword `groups` allows us to define a set of attributes to then be applied to any number of nodes, thus avoiding having to repeat the same exact configuration for each node. This is used in conjunction with the keyword `__clabNodeName__`, that is substituted at compile-time with the actual node name (CE1, PE1, etc.).

### FRR node

The node is based on the image `frr-ssh:10.4.1`, discussed in section 2.2.1. Some file are needed for an FRR node:

- `frr.conf` — Startup configuration file
- `daemons` — Daemon file, used at startup to setup particular daemons (bgpd, ospfd among others)
- `vtysh.conf` — To enable vtysh
- `frr-cfg.sh` — To start the main FRR process, start SSH and possibly setup the management VRF

The node configuration within the topology file is represented in Listing 3.2:

```
1   frr_nodes:
2     env:
3       CLAB_MGMT_VRF: MGMT
4       CLAB_MGMT_GW: 10.255.255.1
5     kind: linux
6     image: frr-ssh:10.4.1
7     binds:
8       - ./config/startup/__clabNodeName__.conf:/etc/frr/frr.conf
9       - ./config/frr/daemons:/etc/frr/daemons
10      - ./config/frr/vtysh.conf:/etc/frr/vtysh.conf
11      - ./config/frr/frr-cfg.sh:/frr-cfg.sh
12    exec:
13      - bash /frr-cfg.sh
```

Listing 3.2: FRR node definition

### Alpine node

The node is based on the image `alpine-ssh:3.19.1`, discussed in section 2.2.2. Some file are needed for an Alpine node:

- `conf.sh` — Startup configuration file
- `alp-cfg.sh` — Start SSH and possibly setup the management VRF

The node configuration within the topology file is represented in Listing 3.3:

```
    alp_nodes:
      kind: linux
      image: alpine-ssh:3.19.1
      binds:
        - ./config/alpine/alp-cfg.sh:/alp-cfg.sh
        - ./config/startup/__clabNodeName__.sh:/conf.sh
      exec:
        - sh /alp-cfg.sh
```

Listing 3.3: Alpine node definition

#### Alpine-Python node

The node is based on the image `python-ssh:3.12-alpine`, discussed in section 2.2.3. Some file are needed for the manager node:

- `conf.sh` — Startup configuration file
- `alp-cfg.sh` — Start SSH and possibly setup the management VRF
- `main.py` — Python file which implements the functionalities required in Task 3.
- `traffic-matrices.json` — Traffic matrix

The node configuration within the topology file is represented in Listing 3.4:

```
    pyt_nodes:
      kind: linux
      image: python-ssh:3.12-alpine
      binds:
        - ./config/alpine/alp-cfg.sh:/alp-cfg.sh
        - ./config/startup/__clabNodeName__.sh:/conf.sh
        - ./config/mngr/main.py:/main.py
        - ./config/mngr/traffic-matrices.json:/traffic-matrices.
            json
      exec:
        - sh /alp-cfg.sh
```

Listing 3.4: Manager node definition

## 3.2 Node configurations

### 3.2.1 FRR nodes

All FRR nodes interfaces have been configured as previously described in sections under 2.3.

**OSPF** The Open Shortest Path First (OSPF) routing protocol has been configured in the central AS (AS 65020): this has been done in order to have all nodes share their loopback addresses with each other, which in turn is needed for establishing **iBGP** sessions using loopback interfaces.
OSPF is configured in single-area mode. Each router (PE1, PE2, GW1, GW2) participates in the core network and advertises its loopback interface, which is configured as a passive interface.

**BGP** All routers have been configured with BGP; For `router-id` the loopback addresses have been used. For the BGP speakers of the AS 65020 a full-mesh topology has been setup. All the BGP relations are represented in the following table:

| Type | Node 1 | Node 1 int. | Node 2 | Node 2 int. |
|---|---|---|---|---|
| eBGP | CE1 | 10.1.1.1 | PE1 | 10.1.1.2 |
| eBGP | CE1 | 10.1.2.1 | PE2 | 10.1.2.2 |
| eBGP | CE2 | 10.2.1.1 | PE1 | 10.2.1.2 |
| eBGP | CE2 | 10.2.2.1 | PE2 | 10.2.2.2 |
| iBGP | PE1 | 192.168.100.11 | PE2 | 192.168.100.12 |
| iBGP | PE1 | 192.168.100.11 | GW1 | 192.168.100.21 |
| iBGP | PE1 | 192.168.100.11 | GW2 | 192.168.100.22 |
| iBGP | PE2 | 192.168.100.12 | GW1 | 192.168.100.21 |
| iBGP | PE2 | 192.168.100.12 | GW2 | 192.168.100.22 |
| iBGP | GW1 | 192.168.100.21 | GW2 | 192.168.100.22 |
| eBGP | GW1 | 172.16.0.1 | UP1 | 172.16.0.2 |
| eBGP | GW2 | 172.16.1.1 | UP2 | 172.16.1.2 |
| eBGP | UP1 | 172.16.2.1 | INT | 172.16.2.2 |
| eBGP | UP2 | 172.16.3.1 | INT | 172.16.3.2 |

Table 3.1: BGP relations

### 3.2.2 Alpine nodes

All Alpine nodes interfaces have been configured as previously described in sections under 2.3. The only additional configuration consists of a default route on the customer hosts (n1, n2, n3 and n4), directed toward their respective routers (CE1, CE2 and INT), which consequently operate as default gateways.

## 3.3 Templates

As required by Task 1 of the project specification, all configuration files are generated with templates. Due to the presence of different node types, each characterized by a different configuration syntax, two distinct templates have been defined: one for FRR-based routers and one for Alpine nodes.

The configuration generation process is based on a single Python script responsible for rendering the templates using the provided input data. The configuration data are stored in the directory `./template/data/*.yaml`, where each file is named according to the corresponding node (i.e., `<node-name>.yaml`). Each data file explicitly specifies the template to be used through the field:

```
type: frr | alpine
```

During execution, the Python script reads this field and automatically selects the appropriate template file, rendering it with the associated data to generate the final device configuration. The content of the Python script (`./template/generator.py`) is shown in Listing 3.5

```
TEMPLATE_FRR = 'template_frr.j2'
TEMPLATE_ALP = 'template_alp.j2'
BASE_DIR = Path(__file__).resolve().parent

# === Load data
data_filename = sys.argv[1]
with open(data_filename) as file:
    data = yaml.safe_load(file)

# Default type: FRR
cont_type = 'FRR'
if 'type' in data:
    if data['type'].lower() == 'alpine':
        cont_type = 'alpine'

# === Load template
environment = Environment(
    loader=FileSystemLoader(str(BASE_DIR)),
    trim_blocks=True,
    lstrip_blocks=True
)
template = None
if cont_type == 'FRR':
    template = environment.get_template(TEMPLATE_FRR)
elif cont_type == 'alpine':
    template = environment.get_template(TEMPLATE_ALP)

# === Generate content
content = template.render(data)

# === Print and save to file

output_dir = BASE_DIR / ".." / "config" / "startup"
output_dir = output_dir.resolve()

config_filename = data['config_filename']
output_path = output_dir / config_filename
output_dir.mkdir(parents=True, exist_ok=True)

with open(output_path, "w") as f:
    f.write(content)
```

Listing 3.5: Template processing

Note that some checks and debug lines have been omitted from this Listing.

**Complete automation** the previously described process creates the configuration file for a given node: this process is easily repeated on all nodes via a simple shell script (`./shell/template.sh`), as represented in Listing 3.6:

```
# Generate startup-config files
for file in template/data/*.yaml; do
    python3 template/generator.py "$file"
    echo "$file" processed
done
```

Listing 3.6: template.sh file

The FRR template is discussed in the following three subsections (*Interfaces, OSPF, BGP*), while the Alpine template is discussed in the last subsection (*Alpine template*).

### 3.3.1 FRR template

#### Interfaces

The template section dealing with interfaces loops over a list of interfaces, which must be called `interfaces`, and for each interface sets:

- IP address — field `ipv4`
- description — field `description`
- working layer — field `switchport` set to `False` for L3, set to `True` for L2

and finally activates it. The part of the template dealing with interfaces is represent in Listing 3.7:

```
{# ===Interfaces === -#}
{% if interfaces -%}
{% for int in interfaces -%}
interface {{int.name}}
    {% if int.ipv4 %}
    ip address {{int.ipv4}}
    {% endif -%}
    {% if int.description %}
    description {{int.description}}
    {% endif %}
    {% if int.switchport == False %}
    no switchport
    {% endif -%}
    no shutdown
!
{% endfor -%}
{% endif -%}
```

Listing 3.7: FRR template - interfaces

An example of a parameters file to be passed, compliant with the previously described part of template, is represented in the following Listing 3.8:

```
interfaces:
- name: lo
  ipv4: 192.168.100.11/32
  switchport: False
  description: LOOPBACK
- name: eth1
  ipv4: 192.168.0.1/29
  switchport: False
  description: to_CORE
```

Listing 3.8: Interfaces parameters

The resulting part of the .conf file is represented in the following Listing 3.9:

```
interface lo
    ip address 192.168.100.11/32
    description LOOPBACK
    no switchport
    no shutdown
!
interface eth1
    ip address 192.168.0.1/29
    description to_CORE
    no switchport
    no shutdown
!
```

Listing 3.9: Configuration example

### OSPF

The template section dealing with OSPF firstly checks if there's a section dedicated to OSPF (field `ospf`): if that field is not present OSPF is not configured. If instead the `ospf` field is present, the template configures the router-id (field `router_id`), then loops over a list of networks (field `networks`) and configures the advertising of those network prefixes, along with their relative area.
Then the template loops over a list of passive interfaces (field `passive_interfaces`) and marks them as passive, with regard to OSPF. The part of the template dealing with OSPF is represented in Listing 3.10:

```
{# === OSPF === -#}
{% if ospf -%}
router ospf
    router-id {{ospf.router_id}}
    {% for net in ospf.networks -%}
    network {{net.network}} area {{net.area}}
    {% endfor %}
!
{% for pi in ospf.passive_interfaces -%}
interface {{pi}}
    ip ospf passive
```

```
{% endfor -%}
{% endif -%}
```

Listing 3.10: FRR template - OSPF

An example of a parameters file to be passed, compliant with the previously described part of template, is represented in the following Listing 3.11:

```
ospf:
  router_id: 192.168.100.11
  passive_interfaces: [lo, eth0, eth2, eth3]
  networks:
  - network: 192.168.0.0/29
    area: 0
  - network: 192.168.100.11/32
    area: 0
```

Listing 3.11: OSPF parameters

The resulting part of the .conf file is represented in the following Listing 3.12:

```
router ospf
    router-id 192.168.100.11
network 192.168.0.0/29 area 0
network 192.168.100.11/32 area 0
!
interface lo
    ip ospf passive
interface eth0
    ip ospf passive
interface eth2
    ip ospf passive
interface eth3
    ip ospf passive
!
```

Listing 3.12: OSPF Configuration example

### BGP

The template section dealing with BGP firstly checks if there's a section dedicated to BGP (field `bgp`): if that field is not present BGP is not configured.
If instead the `bgp` field is present, the template configures the router-id (field `ID`), the current router's ASN (field `AS`) and configures BGP to install ECMP routes among different AS paths of the same length, via the command `bestpath as-path multipath-relax`. Also two other fields are needed for the neighbors configuration:

- `lo_iBGP` — indicates whether to use the loopback interface for iBGP sessions
- `lo_eBGP` — indicates whether to use the loopback interface for eBGP sessions

- `lo` — the loopback interface name

The template loops over a list of BGP neighbors (field `neighbors`) and for each one configures:

- Neighbor IP address — field neighbor.`ipv4`
- Neighbor ASN — if the field neighbor.`AS` is present, that value is used; otherwise it is intended as a **i**BGP session, thus setting the remote-as field to the same ASN as the current router.
- Interface used — if the loopback interface must be used for the current neighbor it is set with the `update-source` command.
- Next-hop modification — for all iBGP peerings the router is configured to overwrite the Next-Hop field by putting its address as Next-Hop, before sending the route advertisement to its iBGP peer; this is done with the `next-hop-self` command

The template then loops over a list of networks (field `networks`) and configure each one of them to be advertised via BGP updates. Finally, the `no bgp ebgp-requires-policy` rule is added: this allows us to let BGP run without needing all BGP updates to have a policy.
The part of the template dealing with BGP is represented in Listing 3.13:

```
{# === BGP === -#}
{% if bgp -%}
router bgp {{bgp.AS}}
    bgp router-id {{bgp.ID}}
    no bgp ebgp-requires-policy
    bgp bestpath as-path multipath-relax
{% for nb in bgp.neighbors -%}
    {% if nb.AS %}
    neighbor {{nb.ipv4}} remote-as {{nb.AS}}
    {% if bgp.lo_eBGP %}
    neighbor {{nb.ipv4}} update-source {{bgp.lo}}
    {% endif -%}
    {% else %}
    neighbor {{nb.ipv4}} remote-as {{bgp.AS}}
    neighbor {{nb.ipv4}} next-hop-self
    {% if bgp.lo_iBGP %}
    neighbor {{nb.ipv4}} update-source {{bgp.lo}}
    {% endif -%}
    {% endif -%}
{% endfor -%}
{% for net in bgp.networks %}
    network {{net.prefix}} mask {{net.mask}}
{% endfor %}
!
{% for nb in bgp.neighbors -%}
clear ip bgp {{nb.ipv4}} soft in
clear ip bgp {{nb.ipv4}} soft out
{% endfor -%}
{% endif -%}
```

```
!
```

Listing 3.13: FRR template - BGP

The template also loops again over the list of neighbors (field `neighbors`) and clears all routes received and sent to each neighbor: this doesn't regard the actual BGP configuration, it is done to enforce a refreshing of all BGP routes in the system, in order to avoiding possible errors in the configuration phase.
An example of a parameters file to be passed, compliant with the previously described part of template, is represented in the following Listing 3.14:

```
bgp:
  AS: 65020
  ID: 192.168.100.11
  lo_iBGP: True
  lo_eBGP: False
  lo: lo

  neighbors:
  - ipv4: 10.1.1.1
    AS: 64501
  - ipv4: 10.2.1.1
    AS: 64502
  - ipv4: 192.168.100.12 # No AS specified -> same AS
  - ipv4: 192.168.100.21
  - ipv4: 192.168.100.22
```

Listing 3.14: BGP parameters

The resulting part of the .conf file is represented in the following Listing 3.15:

```
router bgp 65020
    bgp router-id 192.168.100.11
    neighbor 10.1.1.1 remote-as 64501
    neighbor 10.2.1.1 remote-as 64502
    neighbor 192.168.100.12 remote-as 65020
    neighbor 192.168.100.12 next-hop-self
    neighbor 192.168.100.12 update-source lo
    neighbor 192.168.100.21 remote-as 65020
    neighbor 192.168.100.21 next-hop-self
    neighbor 192.168.100.21 update-source lo
    neighbor 192.168.100.22 remote-as 65020
    neighbor 192.168.100.22 next-hop-self
    neighbor 192.168.100.22 update-source lo
    no bgp ebgp-requires-policy
!
clear ip bgp 10.1.1.1 soft in
clear ip bgp 10.1.1.1 soft out
clear ip bgp 10.2.1.1 soft in
clear ip bgp 10.2.1.1 soft out
clear ip bgp 192.168.100.12 soft in
clear ip bgp 192.168.100.12 soft out
clear ip bgp 192.168.100.21 soft in
clear ip bgp 192.168.100.21 soft out
clear ip bgp 192.168.100.22 soft in
clear ip bgp 192.168.100.22 soft out
```

```
!
```

Listing 3.15: BGP configuration example

### 3.3.2 Alpine template

The template designed for Alpine nodes configures three aspects:

- Interfaces — field `interfaces` — Given a list of interfaces, the IP address (`ipv4`) is configured for each one (`name`)
- Static route — field `ipv4_routes` — All specified routes are added; both the destination (`destination`) and next-hop field are requested (`next_hop`)
- Python — field `python` — Designed for the manager node. It requires a python filename (`file`), which is run in the background and its output is redirected to a log file (`log`); if specified (`messages`), some messages can be displayed via echo message

The template, once compiled, generates a shell file. The template used for Alpine nodes, both standard and Python-Alpine, is shown in Listing 3.16:

```
# === Interfaces
{% if interfaces -%}
{% for int in interfaces -%}
ip link set {{ int.name }} up
{% if int.ipv4 -%}
ip addr add {{ int.ipv4 }} dev {{ int.name }}
{% endif -%}
{% endfor -%}
{% endif %}

{% if ipv4_routes -%}
# === Static routes
{% for route in ipv4_routes -%}
ip route replace {{ route.destination }} via {{ route.next_hop }}
{% endfor -%}
{% endif %}

{% if python -%}
# === Python script
# Run it in background, redirect output to log file
nohup python3 -u {{python.file}} > {{python.log}} 2>&1 &
# Messages
{% for msg in python.msgs -%}
echo {{msg}}
{% endfor -%}
{% endif %}
```

Listing 3.16: Alpine template

An example of a parameters file to be passed, compliant with the previously described template is represented in the following Listing 3.17:

```yaml
type: alpine

interfaces:
- name: eth1
  switchport: False
  ipv4: 192.168.0.5/29

ipv4_routes:
- destination: 0.0.0.0/0
  next_hop: 192.0.2.1

python:
  file: /main.py
  log: /log_file.log
  msgs:
   - "Python code is running in the background"
   - "Output is being redirected to ./log_file.log"
   - "Type tail -f log_file.log to see the live output"
```

Listing 3.17: Alpine parameters

The resulting part of the .conf file is represented in the following Listing 3.18:

```
# === Interfaces
ip link set eth1 up
ip addr add 192.168.0.5/29 dev eth1

# === Static routing
ip route replace 0.0.0.0/0 via 192.0.2.1

# === Python file
# Run it in background, redirect output to log file
nohup python3 -u /main.py > /log_file.log 2>&1 &
# Messages
echo Python code is running in the background
echo Output is being redirected to ./log_file.log
echo Type tail -f log_file.log to see the live output
```

Listing 3.18: Alpine configuration example

## 3.4 Network automation

As per project specification the traffic load balancing via BGP attributes is done only for the two upstream links (GW1-UP1 and GW2-UP2).
The entire network automation procedure is contained inside a python script called `main.py`. The Manager node executes this Python script passed to it with `bind` CONTAINERlab command. The script is executed in launched in background at the network deploy using bash script called `mngr.sh`.
Moreover, a file called `traffic-matrices.json` is passed to the manger as well. In this json file, the traffic estimation (Gb/s) for each route is included. An exmaple of a traffic matrix structure is shown in Listing 3.19:

```
1 { "from": "192.0.2.0/24", "to": "203.0.113.0/25", "traf": 12 },
2 { "from": "198.51.100.0/24", "to": "203.0.113.0/25", "traf": 7 },
3 { "from": "192.0.2.0/24", "to": "203.0.113.128/25", "traf": 5 },
4 { "from": "198.51.100.0/24", "to": "203.0.113.128/25", "traf": 6 },
5 { "from": "203.0.113.0/25", "to": "192.0.2.0/24", "traf": 9 },
6 { "from": "203.0.113.128/25","to": "192.0.2.0/24", "traf": 3 },
7 { "from": "203.0.113.0/25", "to": "198.51.100.0/24", "traf": 6 },
8 { "from": "203.0.113.128/25", "to": "198.51.100.0/24", "traf": 10 }
```

Listing 3.19: A Traffic Matrix

### 3.4.1 Load balancing algorithm

In the Python script, there is an infinite loop. In every iteration, a new traffic matrix is obtained, and a simple greedy algorithm splits the traffic. Only the traffic flowing from the customer networks (CU1 and CU2) towards the Internet is considered: it is the traffic that each customer edge (CE) can forward either to PE1 or to PE2, and that the gateways (GW) can send either to UP1 or to UP2. The greedy algorithm exploits two buckets in order to determine an efficient traffic distribution across the provider edges and the upstream links. The algorithm operates iteratively[1] according to the following steps:

1. Retrieve a route (CU → Internet destination) from the set of available routes.
2. Among the two buckets, select the one with the minimum amount of expected traffic.
3. Insert the destination of the route into the chosen bucket and update the bucket's total expected traffic by adding the traffic associated with the route.
4. If there are still unprocessed routes, return to step 1; otherwise, terminate the algorithm.

An additional constraint is imposed on the route allocation process. Consider the case in which a route from CU1 to a given Internet destination is assigned to a bucket. If a subsequent route from CU2 to the same destination is processed, it must be inserted in the same bucket, and therefore served by the same PE and the same GW. In other words, it is not allowed to assign the route (CU1 → Internet) to GW1 and the route (CU2 → Internet) to GW2, unless additional mechanisms such as traffic engineering or segment routing are employed.
Once the algorithm has been executed, two BGP attributes are exploited, each one acting on a different segment of the path:

- **MED** balances the traffic between the customer edges and the provider edges (CE → PE).

[1] Between every iteration `time.sleep(180)` is called.

- *local_pref* balances the outbound traffic between the gateways and the upstream providers (GW → UP).

In particular, bucket1 is used to determine which routes must be announced with a low MED by PE1 (and with a high MED by PE2), and which routes must have a 200 local-pref in GW1 (to-Internet routes). The opposite goes for bucket2: low MED on PE2, high MED on PE1, and local-pref 200 on GW2. In this way, the traffic towards a destination in bucket1 follows the path CE → PE1 → GW1 → UP1, while the traffic towards a destination in bucket2 follows CE → PE2 → GW2 → UP2.
Note that MED is well suited to the CE → PE segment of the topology: each CE is connected to both PE1 and PE2, which belong to the same AS. Since MED is compared only between paths received from the same neighboring AS, the CE can effectively use it to choose between the two PEs. Inbound traffic (Internet → customer networks) is not explicitly engineered.

### 3.4.2 Local Preference Setting

The *Local Preference* is set using two route-maps, **SET-LP-GW1** and **SET-LP-GW2**, respectively applied on `GW1` and `GW2`. For each entry inside a bucket, an *IP prefix-list* is created, together with a new rule in the corresponding route-map. This rule is applied when a match with the previously defined prefix-list occurs and consists solely in setting the local preference value to 200. If the route map is created in GW1, this has to be applied to the routes received from UP1. The opposite goes for GW2 and UP2.

```
1  def configure_bgp_local_pref(gw_ip, route_map_name, bucket):
2
3      my_up = UP1_IP if gw_ip == GW1_IP else UP2_IP
4
5      cmds = ['configure terminal']
6
7      # rebuild the route-map from scratch
8      cmds.append(f'no route-map {route_map_name}')
9
10     seq = 10
11
12     for prefix in bucket:
13         pl_name = f'PL-{prefix.replace("/", "_")}'
14
15         cmds.append(f'ip prefix-list {pl_name} permit {prefix}')
16         cmds.append(f'route-map {route_map_name} permit {seq}')
17         cmds.append(f' match ip address prefix-list {pl_name}')
18         cmds.append(f' set local-preference {LOCAL_PREF}')
19
20         seq += 10
21
22     # catch all preference
23     cmds.append(f'route-map {route_map_name} permit 999')
24
25     # apply the route-map inbound from the upstream
26     cmds.append(f'router bgp {MY_AS}')
```

```
27      cmds.append(f' neighbor {my_up} route-map {route_map_name} in')
28
29      cmds.append('end')
```

Listing 3.20: Local Preference Route Map Creation

After the command building, this will be passed to the target gateway using `paramiko` library.

### 3.4.3 MED setting

Similarly to local preference, the MED is set using 2 route-maps as well: **SET-MED-PE1** and **SET-MED-PE2**, respectively applied on `PE1` and `PE2`. For each destination, an *IP prefix-list* is created, together with a rule in the route-map that matches the prefix-list and sets the `metric` (MED). Each PE sets a low MED (50) on the destinations of the bucket it has to serve, and a high MED (200) on the destinations of the other bucket[2]. Since the CEs prefer the path with the lowest MED, the traffic towards each destination is sent to the intended PE, while the other PE remains available as a backup. This route-map is applied in the **outbound** direction to all the CE neighbors of the PE.

```
1   def configure_bgp_med(pe_ip, route_map_name, preferred, backup):
2
3       cmds = ['configure terminal']
4
5       # rebuild the route-map from scratch
6       cmds.append(f'no route-map {route_map_name}')
7
8       seq = 10
9
10      for prefix, med in ([(p, MED_PREFERRED) for p in preferred] +
11                          [(p, MED_BACKUP) for p in backup]):
12          pl_name = f'PL-{prefix.replace("/", "_")}'
13
14          cmds.append(f'ip prefix-list {pl_name} permit {prefix}')
15          cmds.append(f'route-map {route_map_name} permit {seq}')
16          cmds.append(f' match ip address prefix-list {pl_name}')
17          cmds.append(f' set metric {med}')
18
19          seq += 10
20
21      # catch-all
22      cmds.append(f'route-map {route_map_name} permit 999')
23
24      # apply the route-map outbound towards the CEs
25      cmds.append(f'router bgp {MY_AS}')
26      for ce_ip in CE_NEIGHBORS[pe_ip]:
27          cmds.append(f' neighbor {ce_ip} route-map {route_map_name}
                out')
28
29      cmds.append('end')
```

[2]Note that in this case bucket1 is served by PE1 (low MED) and backed up on PE2 (high MED), while for bucket2 the opposite holds.

Listing 3.21: MED Route Map Creation

### 3.4.4 BGP Clearing

After route-maps have been created and therefore new rules have to be applied, it is important to clear BGP session with a `soft BGP clear`. On the gateways, both the iBGP peers and the upstream are refreshed with a soft clear in both directions, while on the PEs a `soft out` towards the CEs is enough, since the MED policy is outbound. The script waits for the configuration command to terminate before issuing the clear commands.

```
1  def run_vtysh(ssh, cmds):
2
3      vtysh_cmd = "vtysh \\\n" + " \\\n".join(f'    -c "{c}"' for c
           in cmds)
4
5      stdin, stdout, stderr = ssh.exec_command(vtysh_cmd)
6      # wait for the command to complete
7      stdout.channel.recv_exit_status()
8
9  # on the PEs: only the outbound policy changed
10 for ce_ip in CE_NEIGHBORS[pe_ip]:
11     run_vtysh(ssh, [f'clear ip bgp {ce_ip} soft out'])
12
13 # on the GWs: refresh iBGP peers and upstream
14 for peer_ip in ALL_PEERS:
15     if peer_ip != gw_ip:
16         run_vtysh(ssh, [f'clear ip bgp {peer_ip} soft in',
17                         f'clear ip bgp {peer_ip} soft out'])
18 run_vtysh(ssh, [f'clear ip bgp {my_up} soft in',
19                 f'clear ip bgp {my_up} soft out'])
```

Listing 3.22: BGP clearing

### 3.4.5 Route Maps Clearing

In loop iteration $i$, iteration *i-1* route-map have to be removed to avoid conflicts between rules. To avoid a time window in which no policy is applied, each route-map is removed and recreated within the same `vtysh` call: the first command is `no route-map <name>`, immediately followed by the new rules (see the listings above). The prefix-lists are not removed, since they are created again with the same content at every iteration.

# Chapter 4

# Testing

## 4.1 Network deploy

Here is the network deploy, showed via `containerlab graph` command.

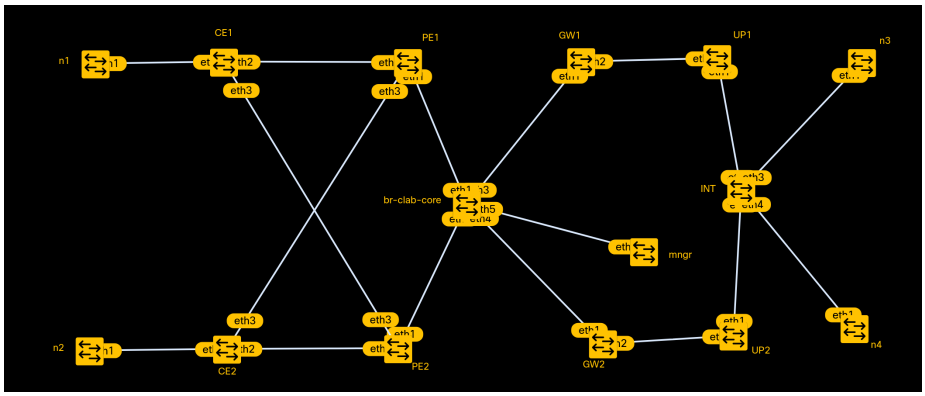



Figure 4.1: containerlab graph

## 4.2 Nodes configuration

In order to show the correct nodes' configuration, here is `show bgp summary` and `show ip route`, executed on various nodes of the network. A `ping` command is called as well, both from n1 and n2, respectively to n3 and n4, as a further confirmation of the correct configuration.

```
CE1# show ip bgp summary

IPv4 Unicast Summary:
BGP router identifier 10.0.0.1, local AS number 64501 VRF default vrf-id 0
BGP table version 15
RIB entries 7, using 896 bytes of memory
Peers 2, using 33 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.1.1.2        4      65020        44        40       15    0    0 00:26:09            3        4 N/A
10.1.2.2        4      65020        43        40       15    0    0 00:26:09            3        4 N/A

Total number of neighbors 2
```

Figure 4.2: CE1 - show ip bgp summary

```
PE1# show ip bgp summary

IPv4 Unicast Summary:
BGP router identifier 192.168.100.11, local AS number 65020 VRF default vrf-id 0
BGP table version 5
RIB entries 7, using 896 bytes of memory
Peers 5, using 83 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.1.1.1        4      64501         8         9        5    0    0 00:01:12            1        4 N/A
10.2.1.1        4      64502         8        10        5    0    0 00:01:13            1        4 N/A
192.168.100.12  4      65020         5         5        5    0    0 00:00:19            2        2 FRRouting/10.4.1_git
192.168.100.21  4      65020         4         5        5    0    0 00:00:29            1        2 FRRouting/10.4.1_git
192.168.100.22  4      65020         7         7        5    0    0 00:00:29            1        2 FRRouting/10.4.1_git

Total number of neighbors 5
```

Figure 4.3: PE1 - show ip bgp summary

```
GW1# show ip bgp summary

IPv4 Unicast Summary:
BGP router identifier 192.168.100.21, local AS number 65020 VRF default vrf-id 0
BGP table version 6
RIB entries 3, using 384 bytes of memory
Peers 4, using 66 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
172.16.0.2      4      65541        36        39        6    0    0 00:04:13            2        2 N/A
192.168.100.11  4      65020        20        17        6    0    0 00:03:30            0        2 FRRouting/10.4.1_git
192.168.100.12  4      65020        20        17        6    0    0 00:03:25            0        2 FRRouting/10.4.1_git
192.168.100.22  4      65020        24        23        6    0    0 00:03:30            0        2 FRRouting/10.4.1_git

Total number of neighbors 4
```

Figure 4.4: GW1 - show ip bgp summary

```
UP1# show ip bgp summary

IPv4 Unicast Summary:
BGP router identifier 172.31.0.1, local AS number 65541 VRF default vrf-id 0
BGP table version 36
RIB entries 7, using 896 bytes of memory
Peers 2, using 33 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
172.16.0.1      4      65020       179       167       36    0    0 00:24:46            1        4 N/A
172.16.2.2      4      65551        57        57       36    0    0 00:24:46            3        4 N/A

Total number of neighbors 2
```

Figure 4.5: UP1 - show ip bgp summary

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n1 sh
/ # ping 203.0.113.10
PING 203.0.113.10 (203.0.113.10): 56 data bytes
64 bytes from 203.0.113.10: seq=0 ttl=59 time=0.186 ms
64 bytes from 203.0.113.10: seq=1 ttl=59 time=0.454 ms
64 bytes from 203.0.113.10: seq=2 ttl=59 time=0.281 ms
64 bytes from 203.0.113.10: seq=3 ttl=59 time=0.331 ms
```

Figure 4.6: ping - from n1, to n3

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n2 sh
/ # ping 203.0.113.138
PING 203.0.113.138 (203.0.113.138): 56 data bytes
64 bytes from 203.0.113.138: seq=0 ttl=59 time=0.243 ms
64 bytes from 203.0.113.138: seq=1 ttl=59 time=0.308 ms
64 bytes from 203.0.113.138: seq=2 ttl=59 time=0.311 ms
64 bytes from 203.0.113.138: seq=3 ttl=59 time=0.307 ms
```

Figure 4.7: ping - from n2, to n4

```
PE1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
C>* 10.1.1.0/30 is directly connected, eth2, weight 1, 00:01:09
L>* 10.1.1.2/32 is directly connected, eth2, weight 1, 00:01:09
C>* 10.2.1.0/30 is directly connected, eth3, weight 1, 00:01:09
L>* 10.2.1.2/32 is directly connected, eth3, weight 1, 00:01:09
B>* 192.0.2.0/24 [20/0] via 10.1.1.1, eth2, weight 1, 00:01:08
O   192.168.0.0/29 [110/10] is directly connected, eth1, weight 1, 00:00:20
C>* 192.168.0.0/29 is directly connected, eth1, weight 1, 00:01:09
L>* 192.168.0.1/32 is directly connected, eth1, weight 1, 00:01:09
O   192.168.100.11/32 [110/0] is directly connected, lo, weight 1, 00:01:09
L * 192.168.100.11/32 is directly connected, lo, weight 1, 00:01:09
C>* 192.168.100.11/32 is directly connected, lo, weight 1, 00:01:09
O>* 192.168.100.12/32 [110/10] via 192.168.0.2, eth1, weight 1, 00:00:20
O>* 192.168.100.21/32 [110/10] via 192.168.0.3, eth1, weight 1, 00:00:20
O>* 192.168.100.22/32 [110/10] via 192.168.0.4, eth1, weight 1, 00:00:20
B>* 198.51.100.0/24 [20/0] via 10.2.1.1, eth3, weight 1, 00:01:07
B>  203.0.113.0/25 [200/0] via 192.168.100.21 (recursive), weight 1, 00:00:18
  *                          via 192.168.0.3, eth1, weight 1, 00:00:18
                           via 192.168.100.22 (recursive), weight 1, 00:00:18
  *                          via 192.168.0.4, eth1, weight 1, 00:00:18
B>  203.0.113.128/25 [200/0] via 192.168.100.21 (recursive), weight 1, 00:00:18
  *                            via 192.168.0.3, eth1, weight 1, 00:00:18
                             via 192.168.100.22 (recursive), weight 1, 00:00:18
  *                            via 192.168.0.4, eth1, weight 1, 00:00:18
```

Figure 4.8: PE1 - show ip route

```
GW1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
C>* 172.16.0.0/30 is directly connected, eth2, weight 1, 00:02:09
L>* 172.16.0.1/32 is directly connected, eth2, weight 1, 00:02:09
B>  192.0.2.0/24 [200/0] via 192.168.100.11 (recursive), weight 1, 00:01:17
  *                        via 192.168.0.1, eth1, weight 1, 00:01:17
                         via 192.168.100.12 (recursive), weight 1, 00:01:17
  *                        via 192.168.0.2, eth1, weight 1, 00:01:17
O   192.168.0.0/29 [110/10] is directly connected, eth1, weight 1, 00:01:19
C>* 192.168.0.0/29 is directly connected, eth1, weight 1, 00:02:09
L>* 192.168.0.3/32 is directly connected, eth1, weight 1, 00:02:09
O>* 192.168.100.11/32 [110/10] via 192.168.0.1, eth1, weight 1, 00:01:19
O>* 192.168.100.12/32 [110/10] via 192.168.0.2, eth1, weight 1, 00:01:19
O   192.168.100.21/32 [110/0] is directly connected, lo, weight 1, 00:02:09
L * 192.168.100.21/32 is directly connected, lo, weight 1, 00:02:09
C>* 192.168.100.21/32 is directly connected, lo, weight 1, 00:02:09
O>* 192.168.100.22/32 [110/10] via 192.168.0.4, eth1, weight 1, 00:01:19
B>  198.51.100.0/24 [200/0] via 192.168.100.11 (recursive), weight 1, 00:01:17
  *                           via 192.168.0.1, eth1, weight 1, 00:01:17
                            via 192.168.100.12 (recursive), weight 1, 00:01:17
  *                           via 192.168.0.2, eth1, weight 1, 00:01:17
B>* 203.0.113.0/25 [20/0] via 172.16.0.2, eth2, weight 1, 00:02:07
B>* 203.0.113.128/25 [20/0] via 172.16.0.2, eth2, weight 1, 00:02:07
```

Figure 4.9: GW1 - show ip route

```
CE1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
L * 10.0.0.1/32 is directly connected, lo, weight 1, 00:01:30
C>* 10.0.0.1/32 is directly connected, lo, weight 1, 00:01:30
C>* 10.1.1.0/30 is directly connected, eth2, weight 1, 00:01:30
L>* 10.1.1.1/32 is directly connected, eth2, weight 1, 00:01:30
C>* 10.1.2.0/30 is directly connected, eth3, weight 1, 00:01:30
L>* 10.1.2.1/32 is directly connected, eth3, weight 1, 00:01:30
C>* 192.0.2.0/24 is directly connected, eth1, weight 1, 00:01:30
L>* 192.0.2.1/32 is directly connected, eth1, weight 1, 00:01:30
B>* 198.51.100.0/24 [20/0] via 10.1.1.2, eth2, weight 1, 00:01:28
  *                        via 10.1.2.2, eth3, weight 1, 00:01:28
B>* 203.0.113.0/25 [20/0] via 10.1.1.2, eth2, weight 1, 00:00:38
  *                       via 10.1.2.2, eth3, weight 1, 00:00:38
B>* 203.0.113.128/25 [20/0] via 10.1.1.2, eth2, weight 1, 00:00:38
  *                         via 10.1.2.2, eth3, weight 1, 00:00:38
```

Figure 4.10: CE1 - show ip route

```
UP1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
C>* 172.16.0.0/30 is directly connected, eth2, weight 1, 00:01:47
L>* 172.16.0.2/32 is directly connected, eth2, weight 1, 00:01:47
C>* 172.16.2.0/24 is directly connected, eth1, weight 1, 00:01:47
L>* 172.16.2.1/32 is directly connected, eth1, weight 1, 00:01:47
L * 172.31.0.1/32 is directly connected, lo, weight 1, 00:01:47
C>* 172.31.0.1/32 is directly connected, lo, weight 1, 00:01:47
B>* 192.0.2.0/24 [20/0] via 172.16.0.1, eth2, weight 1, 00:00:55
B>* 198.51.100.0/24 [20/0] via 172.16.0.1, eth2, weight 1, 00:00:55
B>* 203.0.113.0/25 [20/0] via 172.16.2.2, eth1, weight 1, 00:01:45
B>* 203.0.113.128/25 [20/0] via 172.16.2.2, eth1, weight 1, 00:01:45
```

Figure 4.11: UP1 - show ip route

## 4.3 Automation testing

**Network Legend**

- **Customer Network 1 (CuNet1)**: 192.0.2.0/24
- **Customer Network 2 (CuNet2)**: 198.51.100.0/24
- **External Network 1 (ExtNet1)**: 203.0.113.0/25
- **External Network 2 (ExtNet2)**: 203.0.113.128/25

**Traffic Matrix 1**

| From / To | CuNet1 | CuNet2 | ExtNet1 | ExtNet2 |
|---|---|---|---|---|
| **CuNet1** | – | – | 12 | 5 |
| **CuNet2** | – | – | 7 | 6 |
| **ExtNet1** | 9 | 6 | – | – |
| **ExtNet2** | 3 | 10 | – | – |

**Traffic Matrix 2**

| From / To | CuNet1 | CuNet2 | ExtNet1 | ExtNet2 |
|---|---|---|---|---|
| **CuNet1** | – | – | 8 | 3 |
| **CuNet2** | – | – | 25 | 6 |
| **ExtNet1** | 5 | 4 | – | – |
| **ExtNet2** | 2 | 7 | – | – |

The greedy algorithm will provide the following result, w.r.t. the previous matrices:

**Matrix 1 Buckets**

- **Bucket 1**: ExtNet1, CuNet2
- **Bucket 2**: ExtNet2, CuNet1

**Matrix 2 Buckets**

- **Bucket 1**: ExtNet1
- **Bucket 2**: ExtNet2, CuNet1, CuNet2

From the following images, we can clearly see the bgp table and routing table change accordingly to the bucket.

### 4.3.1 Resulting tables

#### Matrix 1 - resulting tables

```
PE1# show ip bgp
BGP table version is 5, local router ID is 192.168.100.11, vrf id 0
Default local pref 100, local AS 65020
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     10.1.1.1                 0             0 64501 i
 * i                  192.168.100.12           0    100      0 64501 i
 *>  198.51.100.0/24  10.2.1.1                 0             0 64502 i
 * i                  192.168.100.12           0    100      0 64502 i
 *>i 203.0.113.0/25   192.168.100.21                200      0 65541 65551 i
 *>i 203.0.113.128/25 192.168.100.22                200      0 65542 65551 i

Displayed 4 routes and 6 total paths
```

Figure 4.12: PE1 - Traffic matrix 1 - show ip bgp

```
INT# show ip bgp
BGP table version is 4, local router ID is 203.0.114.1, vrf id 0
Default local pref 100, local AS 65551
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     172.16.3.1                               0 65542 65020 64501 i
 *>  198.51.100.0/24  172.16.2.1                               0 65541 65020 64502 i
 *>  203.0.113.0/25   0.0.0.0                  0         32768 i
 *>  203.0.113.128/25 0.0.0.0                  0         32768 i

Displayed 4 routes and 4 total paths
```

Figure 4.13: INT - Traffic matrix 1 - show ip bgp

```
PE1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
C>* 10.1.1.0/30 is directly connected, eth2, weight 1, 00:01:30
L>* 10.1.1.2/32 is directly connected, eth2, weight 1, 00:01:30
C>* 10.2.1.0/30 is directly connected, eth3, weight 1, 00:01:30
L>* 10.2.1.2/32 is directly connected, eth3, weight 1, 00:01:30
B>* 192.0.2.0/24 [20/0] via 10.1.1.1, eth2, weight 1, 00:01:27
O   192.168.0.0/29 [110/10] is directly connected, eth1, weight 1, 00:00:44
C>* 192.168.0.0/29 is directly connected, eth1, weight 1, 00:01:30
L>* 192.168.0.1/32 is directly connected, eth1, weight 1, 00:01:30
O   192.168.100.11/32 [110/0] is directly connected, lo, weight 1, 00:01:30
L * 192.168.100.11/32 is directly connected, lo, weight 1, 00:01:30
C>* 192.168.100.11/32 is directly connected, lo, weight 1, 00:01:30
O>* 192.168.100.12/32 [110/10] via 192.168.0.2, eth1, weight 1, 00:00:39
O>* 192.168.100.21/32 [110/10] via 192.168.0.3, eth1, weight 1, 00:00:34
O>* 192.168.100.22/32 [110/10] via 192.168.0.4, eth1, weight 1, 00:00:44
B>* 198.51.100.0/24 [20/0] via 10.2.1.1, eth3, weight 1, 00:01:28
B>  203.0.113.0/25 [200/0] via 192.168.100.21 (recursive), weight 1, 00:00:33
  *                          via 192.168.0.3, eth1, weight 1, 00:00:33
B>  203.0.113.128/25 [200/0] via 192.168.100.22 (recursive), weight 1, 00:00:43
  *                            via 192.168.0.4, eth1, weight 1, 00:00:43
```

Figure 4.14: PE1 - Traffic matrix 1 - show ip route

```
INT# show ip bgp
BGP table version is 4, local router ID is 203.0.114.1, vrf id 0
Default local pref 100, local AS 65551
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     172.16.3.1                             0 65542 65020 64501 i
 *>  198.51.100.0/24  172.16.2.1                             0 65541 65020 64502 i
 *>  203.0.113.0/25   0.0.0.0                  0         32768 i
 *>  203.0.113.128/25 0.0.0.0                  0         32768 i

Displayed 4 routes and 4 total paths
```

Figure 4.15: INT - Traffic matrix 1 - show ip route

### Matrix 2 - resulting tables

```
PE1# show ip bgp
BGP table version is 4, local router ID is 192.168.100.11, vrf id 0
Default local pref 100, local AS 65020
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     10.1.1.1                 0             0 64501 i
 * i                  192.168.100.12           0    100      0 64501 i
 *>  198.51.100.0/24  10.2.1.1                 0             0 64502 i
 * i                  192.168.100.12           0    100      0 64502 i
 *>i 203.0.113.0/25   192.168.100.21                200      0 65541 65551 i
 *>i 203.0.113.128/25 192.168.100.22                200      0 65542 65551 i

Displayed 4 routes and 6 total paths
```

Figure 4.16: PE1 - Traffic matrix 2 - show ip bgp

```
INT# show ip bgp
BGP table version is 8, local router ID is 203.0.114.1, vrf id 0
Default local pref 100, local AS 65551
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     172.16.3.1                             0 65542 65020 64501 i
 *>  198.51.100.0/24  172.16.3.1                             0 65542 65020 64502 i
 *>  203.0.113.0/25   0.0.0.0                  0         32768 i
 *>  203.0.113.128/25 0.0.0.0                  0         32768 i

Displayed 4 routes and 4 total paths
```

Figure 4.17: INT - Traffic matrix 2 - show ip bgp

```
PE1# show ip route
Codes: K - kernel route, C - connected, L - local, S - static,
       R - RIP, O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric, t - Table-Direct,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

IPv4 unicast VRF default:
C>* 10.1.1.0/30 is directly connected, eth2, weight 1, 00:04:08
L>* 10.1.1.2/32 is directly connected, eth2, weight 1, 00:04:08
C>* 10.2.1.0/30 is directly connected, eth3, weight 1, 00:04:08
L>* 10.2.1.2/32 is directly connected, eth3, weight 1, 00:04:08
B>* 192.0.2.0/24 [20/0] via 10.1.1.1, eth2, weight 1, 00:04:06
O   192.168.0.0/29 [110/10] is directly connected, eth1, weight 1, 00:03:24
C>* 192.168.0.0/29 is directly connected, eth1, weight 1, 00:04:08
L>* 192.168.0.1/32 is directly connected, eth1, weight 1, 00:04:08
O   192.168.100.11/32 [110/0] is directly connected, lo, weight 1, 00:04:08
L * 192.168.100.11/32 is directly connected, lo, weight 1, 00:04:08
C>* 192.168.100.11/32 is directly connected, lo, weight 1, 00:04:08
O>* 192.168.100.12/32 [110/10] via 192.168.0.2, eth1, weight 1, 00:03:19
O>* 192.168.100.21/32 [110/10] via 192.168.0.3, eth1, weight 1, 00:03:24
O>* 192.168.100.22/32 [110/10] via 192.168.0.4, eth1, weight 1, 00:03:24
B>* 198.51.100.0/24 [20/0] via 10.2.1.1, eth3, weight 1, 00:04:06
B>  203.0.113.0/25 [200/0] via 192.168.100.21 (recursive), weight 1, 00:03:17
  *                          via 192.168.0.3, eth1, weight 1, 00:03:17
B>  203.0.113.128/25 [200/0] via 192.168.100.22 (recursive), weight 1, 00:03:17
  *                            via 192.168.0.4, eth1, weight 1, 00:03:17
```

Figure 4.18: PE1 - Traffic matrix 2 - show ip route

```
INT# show ip bgp
BGP table version is 8, local router ID is 203.0.114.1, vrf id 0
Default local pref 100, local AS 65551
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  192.0.2.0/24     172.16.3.1                             0 65542 65020 64501 i
 *>  198.51.100.0/24  172.16.3.1                             0 65542 65020 64502 i
 *>  203.0.113.0/25   0.0.0.0                  0         32768 i
 *>  203.0.113.128/25 0.0.0.0                  0         32768 i

Displayed 4 routes and 4 total paths
```

Figure 4.19: INT - Traffic matrix 2 - show ip route

### 4.3.2 Traceroute examples

Finally, a comparision between `traceroute` commands. The following images show two uses of `traceroute`, the first one is launched when the current traffic matrix is matrix1, the second one when matrix2 is used.
In order, there are the following:

- n1 - n3
- n2 - n4
- n3 - n1
- n4 - n2

**n1 to n3**

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n1 sh
/ # traceroute 203.0.113.10
traceroute to 203.0.113.10 (203.0.113.10), 30 hops max, 46 byte packets
 1  192.0.2.1 (192.0.2.1)  0.009 ms  0.037 ms  0.011 ms
 2  10.1.1.2 (10.1.1.2)  0.012 ms  0.037 ms  0.007 ms
 3  192.168.0.3 (192.168.0.3)  0.012 ms  0.052 ms  0.007 ms
 4  172.16.2.1 (172.16.2.1)  0.011 ms  0.058 ms  0.025 ms
 5  172.16.3.2 (172.16.3.2)  0.007 ms  0.030 ms  0.008 ms
 6  203.0.113.10 (203.0.113.10)  0.012 ms  0.031 ms  0.030 ms
/ # traceroute 203.0.113.10
traceroute to 203.0.113.10 (203.0.113.10), 30 hops max, 46 byte packets
 1  192.0.2.1 (192.0.2.1)  0.008 ms  0.062 ms  0.014 ms
 2  10.1.1.2 (10.1.1.2)  0.008 ms  0.031 ms  0.010 ms
 3  192.168.0.3 (192.168.0.3)  0.008 ms  0.039 ms  0.007 ms
 4  172.16.2.1 (172.16.2.1)  0.007 ms  0.030 ms  0.009 ms
 5  172.16.3.2 (172.16.3.2)  0.006 ms  0.030 ms  0.007 ms
 6  203.0.113.10 (203.0.113.10)  0.008 ms  0.064 ms  0.014 ms
```

Figure 4.20: Traceroute, from n1 to n3

- Matrix1: n1, CE1, PE1, GW1, UP1, INT, n3
- Matrix2: n1, CE1, PE1, GW1, UP1, INT, n3

**n2 to n4**

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n2 sh
/ # traceroute 203.0.113.138
traceroute to 203.0.113.138 (203.0.113.138), 30 hops max, 46 byte packets
 1  198.51.100.1 (198.51.100.1)  0.014 ms  0.039 ms  0.007 ms
 2  10.2.2.2 (10.2.2.2)  0.052 ms  0.037 ms  0.010 ms
 3  192.168.0.4 (192.168.0.4)  0.007 ms  0.032 ms  0.019 ms
 4  172.16.3.1 (172.16.3.1)  0.010 ms  0.063 ms  0.008 ms
 5  172.16.2.2 (172.16.2.2)  0.006 ms  0.033 ms  0.010 ms
 6  203.0.113.138 (203.0.113.138)  0.013 ms  0.028 ms  0.012 ms
/ # traceroute 203.0.113.138
traceroute to 203.0.113.138 (203.0.113.138), 30 hops max, 46 byte packets
 1  198.51.100.1 (198.51.100.1)  0.009 ms  0.084 ms  0.022 ms
 2  10.2.2.2 (10.2.2.2)  0.017 ms  0.029 ms  0.045 ms
 3  192.168.0.4 (192.168.0.4)  0.009 ms  0.030 ms  0.008 ms
 4  172.16.1.2 (172.16.1.2)  0.006 ms  0.027 ms  0.008 ms
 5  172.16.3.2 (172.16.3.2)  0.046 ms  0.030 ms  0.008 ms
 6  203.0.113.138 (203.0.113.138)  0.008 ms  0.034 ms  0.014 ms
```

Figure 4.21: Traceroute, from n2 to n4

- Matrix1: n2, CE2, PE2, GW2, UP2, INT, n4

- Matrix2: n2, CE2, PE2, GW2, UP2, INT, n4

**n3 to n1**

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n3 sh
/ # traceroute 192.0.2.10
traceroute to 192.0.2.10 (192.0.2.10), 30 hops max, 46 byte packets
 1  203.0.113.1 (203.0.113.1)  0.019 ms  0.039 ms  0.032 ms
 2  172.16.3.1 (172.16.3.1)  0.007 ms  0.024 ms  0.005 ms
 3  192.168.0.4 (192.168.0.4)  0.005 ms  0.034 ms  0.011 ms
 4  192.168.0.2 (192.168.0.2)  0.007 ms  0.032 ms  0.008 ms
 5  10.1.2.1 (10.1.2.1)  0.008 ms  0.037 ms  0.006 ms
 6  192.0.2.10 (192.0.2.10)  0.006 ms  0.025 ms  0.007 ms
/ # traceroute 192.0.2.10
traceroute to 192.0.2.10 (192.0.2.10), 30 hops max, 46 byte packets
 1  203.0.113.1 (203.0.113.1)  0.025 ms  0.049 ms  0.007 ms
 2  172.16.3.1 (172.16.3.1)  0.006 ms  0.038 ms  0.010 ms
 3  192.168.0.4 (192.168.0.4)  0.007 ms  0.041 ms  0.007 ms
 4  192.168.0.2 (192.168.0.2)  0.007 ms  0.047 ms  0.019 ms
 5  10.1.2.1 (10.1.2.1)  0.015 ms  0.031 ms  0.013 ms
 6  192.0.2.10 (192.0.2.10)  0.016 ms  0.031 ms  0.019 ms
```

Figure 4.22: Traceroute, from n3 to n1

- Matrix1: n3, INT, UP2, GW2, PE2, CE1, n1
- Matrix2: n3, INT, UP2, GW2, PE2, CE1, n1

**n4 to n2**

```
lorenzo@Lorenzo-Virtual-Machine:~/Network-Provisioning-Automation$ docker exec -it acn-prj-n4 sh
/ # traceroute 198.51.100.10
traceroute to 198.51.100.10 (198.51.100.10), 30 hops max, 46 byte packets
 1  203.0.113.129 (203.0.113.129)  0.017 ms  0.047 ms  0.008 ms
 2  172.16.2.1 (172.16.2.1)  0.009 ms  0.032 ms  0.016 ms
 3  192.168.0.3 (192.168.0.3)  0.007 ms  0.027 ms  0.007 ms
 4  192.168.0.1 (192.168.0.1)  0.006 ms  0.023 ms  0.006 ms
 5  10.2.1.1 (10.2.1.1)  0.006 ms  0.059 ms  0.010 ms
 6  198.51.100.10 (198.51.100.10)  0.032 ms  0.022 ms  0.019 ms
/ # traceroute 198.51.100.10
traceroute to 198.51.100.10 (198.51.100.10), 30 hops max, 46 byte packets
 1  203.0.113.129 (203.0.113.129)  0.010 ms  0.046 ms  0.010 ms
 2  172.16.3.1 (172.16.3.1)  0.016 ms  0.033 ms  0.005 ms
 3  172.16.1.1 (172.16.1.1)  0.007 ms  0.024 ms  0.006 ms
 4  192.168.0.1 (192.168.0.1)  0.006 ms  0.068 ms  0.014 ms
 5  10.2.1.1 (10.2.1.1)  0.035 ms  0.036 ms  0.010 ms
 6  198.51.100.10 (198.51.100.10)  0.010 ms  0.111 ms  0.011 ms
```

Figure 4.23: Traceroute, from n4 to n2

- Matrix1: n4, INT, UP1, GW1, PE1, CE2, n2
- Matrix2: n4, INT, UP2, GW2, PE1, CE2, n2

# Chapter 5

# Conclusion

## 5.1 Results

The project successfully achieves its primary objectives:

- automating the generation of network configuration files from YAML descriptions using **Jinja2**;
- deploying the network through **containerlab** based on the automatically generated configuration files;
- implementing an automated load balancing mechanism by exploiting **BGP** features such as *local_preference* and route filtering.

## 5.2 Future work

Despite these positive results, several aspects of the project can be further improved, particularly with respect to the load balancing strategy.

**Route maps** In the current implementation, route maps are periodically removed and re-created every three minutes. While this approach is sufficient to demonstrate the automation mechanism and its effectiveness within the scope of this project, it is not efficient and would be unsuitable in a real-world production environment. In practical scenarios, policy updates would typically occur less frequently, and route maps could be modified in place rather than being entirely removed and re-created, thus reducing control-plane overhead and potential instability.

**BGP limitations** Furthermore, relying only on BGP for load balancing introduces intrinsic limitations. BGP performs routing decisions at the prefix level rather than at the flow level and lacks native awareness of network congestion and link utilization. To overcome these limitations, the proposed solution

could be enhanced by integrating more advanced mechanisms such as **Traffic Engineering** and **Segment Routing**. These technologies enable explicit control over flows (not only prefix based as BGP), improving load balancing flexibility and providing faster and more reliable reactions to network faults and congestion.

**Expansion** The current approach used to achieve a form of load balancing across the two upstream links can be replicated to balance traffic on the customer side of the network. We chose not to implement this as only the upstream links were required and it would be conceptually very similar, just increasing the complexity of the algorithm.